Lab 02 – How the Code Fits Together

Structure from Motion & RANSAC line fitting

How to read this document

- The SfM part has **one class** (`Image`). Everything else is **plain functions** that pass NumPy arrays, dicts and `Image` objects around. `sfm.py` is the script that calls them in order.
- Grey boxes are provided code. Orange boxes contain a TODO and are labelled with the task number from the handout.
- Blue text next to a box is what that step *produces*, with array shapes. N = number of 3D points, M = number of matches, K_i = number of keypoints in image i.
- Run `sfm.py` from inside `code/`: it loads data from the relative path `../data`.

Contents

1 The big picture: what `sfm.py` does **2**

2 Files and who imports whom **3**

3 The only class: `Image` **3**

4 How the data structures point at each other **4**

5 Zoom-in on each task **4**

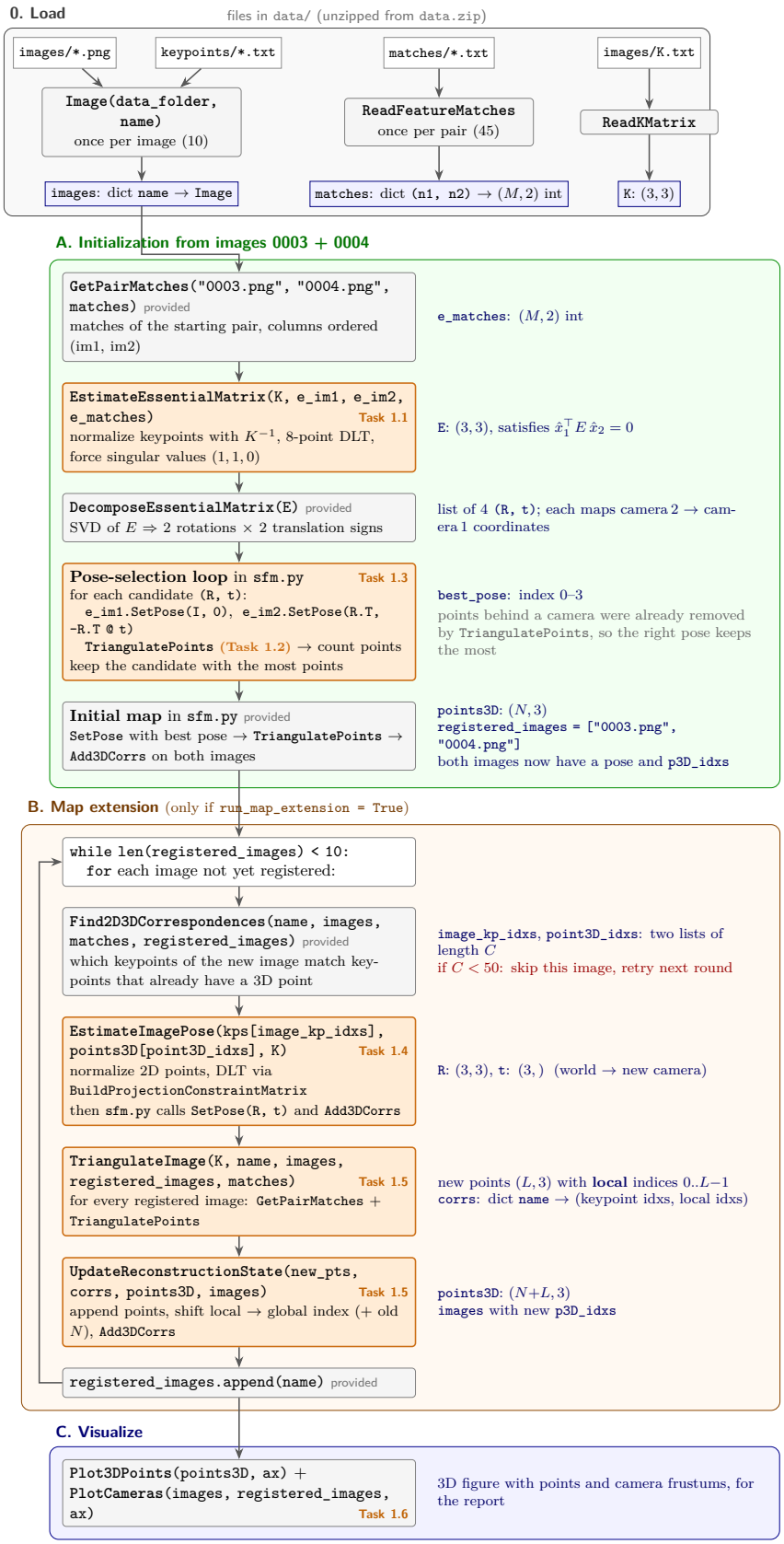
6 Function reference **6**

7 Conventions to keep straight **6**

8 Part 2: `line_fitting.py` **7**

1 The big picture: what `sfm.py` does

`main()` in `sfm.py` runs in four stages: **load** the data, **initialize** the map from two images, **extend** it one image at a time, and **plot** the result. Stage C only runs when `run_map_extension = True`.



The full SfM pipeline in `sfm.py`. `K`, `images` and `matches` are used by almost every step, so only the first arrow out of Stage 0 is drawn.

2 Files and who imports whom

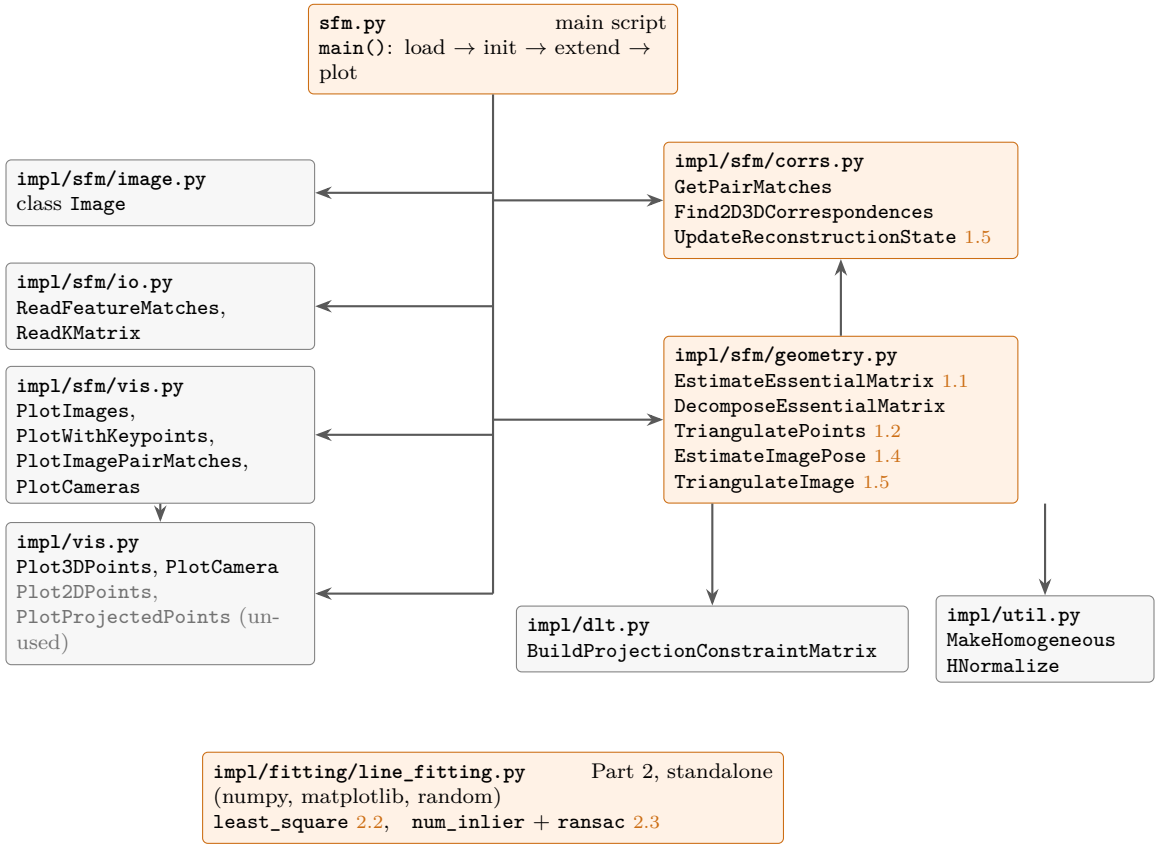


Figure 1: Arrows mean "imports from". Orange modules contain TODOs. Part 2 has no connection to the SfM code.

3 The only class: `Image`

An `Image` bundles everything known about one photo: its pixels, its keypoints, its pose once registered, and which keypoints have a 3D point.

class `Image` (`impl/sfm/image.py`)	
`name`: str, e.g. `"0003.png"`	
`image`: $(H, W, 3)$ float pixel array	(only used for plotting)
`kps`: $(K_i, 2)$ float, row j = pixel $[x, y]$ of keypoint j	
`p3D_idxs`: dict `{keypoint index: 3D point index}`	starts empty
`R`: $(3, 3)$, `t`: $(3,)$	exist only after `SetPose`
`__init__(data_folder, name)`	loads the photo and its keypoint file
`SetPose(R, t)`	store pose (world → camera)
`Pose()` → `(R, t)`	
`Add3DCorrs(kp_idxs, p3D_idxs)`	two equal-length lists; fills the dict
`GetPoint3DIdx(kp_idx)` → int	3D point index, or -1 if none
`NumObserved()` → int	number of keypoints with a 3D point

Note: `t` must be a flat vector of shape $(3,)$, not $(3, 1)$. `TriangulatePoints` calls `np.expand_dims(t, 1)` itself.

4 How the data structures point at each other

This is the part the handout leaves implicit. Everything is connected through **integer indices**:

- A **keypoint index** is a row number in `image.kps`.
- A **match** is a row `[i, j]` of `matches[(n1, n2)]`: keypoint i in image `n1` and keypoint j in image `n2` show the same physical point. Keys are always sorted (`n1 < n2`); `GetPairMatches(a, b, matches)` flips the columns if you ask for the pair in the other order.
- A **3D point index** is a row number in the global `points3D` array.
- `image.p3D_idxs` links the two: keypoint index → 3D point index.

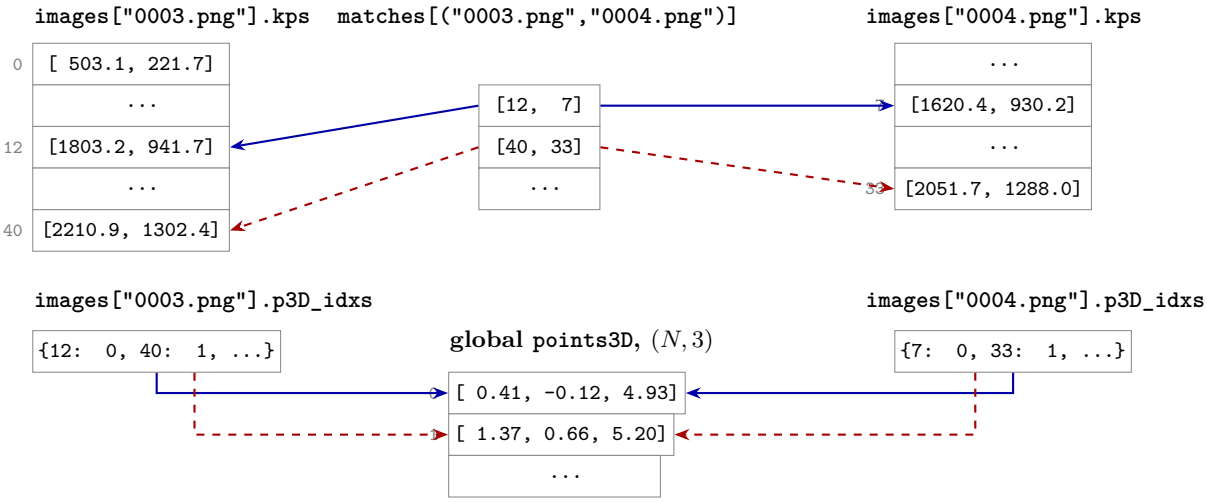
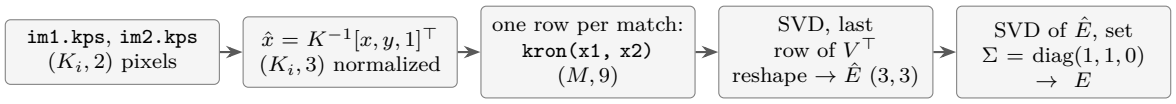


Figure 2: Two example correspondences (blue and red). The first match row says keypoint 12 in 0003 and keypoint 7 in 0004 are the same point. After triangulation, both images' `p3D_idxs` send that keypoint to row 0 of `points3D`. Numbers are illustrative.

Why this matters. `Find2D3DCorrespondences` walks exactly this chain: new image keypoint $\xrightarrow{\text{match}}$ registered image keypoint $\xrightarrow{\text{p3D_idxs}}$ 3D point. If `p3D_idxs` holds wrong indices (e.g. local instead of global in Task 1.5), new images get registered against the wrong 3D points.

5 Zoom-in on each task

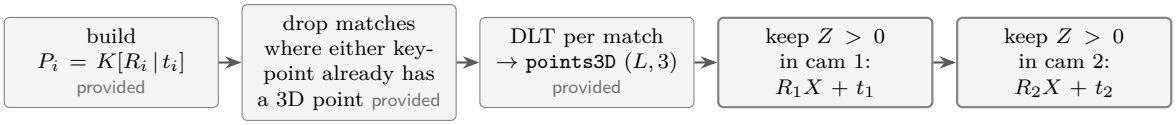
Task 1.1 – `EstimateEssentialMatrix(K, im1, im2, matches)` `geometry.py`



Ends with an `assert` that $|\hat{x}_1^\top E\, \hat{x}_2| < 0.01$ for every match.

Task 1.2 – `TriangulatePoints(K, im1, im2, matches)` `geometry.py`

Requires both images to have a pose already.



Returns `points3D` $(L, 3)$, `im1_corrs` $(L,)$, `im2_corrs` $(L,)$: row k of each belongs to the same point. Use **one mask** for all three arrays.

Task 1.3 – choosing the pose `sfm.py`

Convention. The decomposition returns (R, t) with $x_{\text{cam1}} = R\,x_{\text{cam2}} + t$. `SetPose` wants world → camera, and the world frame is camera 1. So:

$$\text{camera 1: } (I,\ \mathbf{0}) \qquad \text{camera 2: } (R^\top,\ -R^\top t)$$

Try all four candidates, call `TriangulatePoints` for each, and keep the one returning the most points. Then set that pose for real; the provided code triangulates once more and calls `Add3DCorrs` with indices `0..N-1`.

Task 1.4 – `EstimateImagePose(points2D, points3D, K)` `geometry.py`

Inputs are C matched pairs: `points2D` $(C, 2)$ pixels and `points3D` $(C, 3)$. Only the normalization is missing. With $\hat{x} = K^{-1}x$, the projection becomes $\hat{x} \sim [R\,|\,t]X$, so DLT solves for $[R\,|\,t]$ directly. The provided code then extracts R (closest rotation) and t (from the camera centre). `BuildProjectionConstraintMatrix` only reads columns 0 and 1 of the 2D points.

Task 1.5 – `TriangulateImage` + `UpdateReconstructionState`

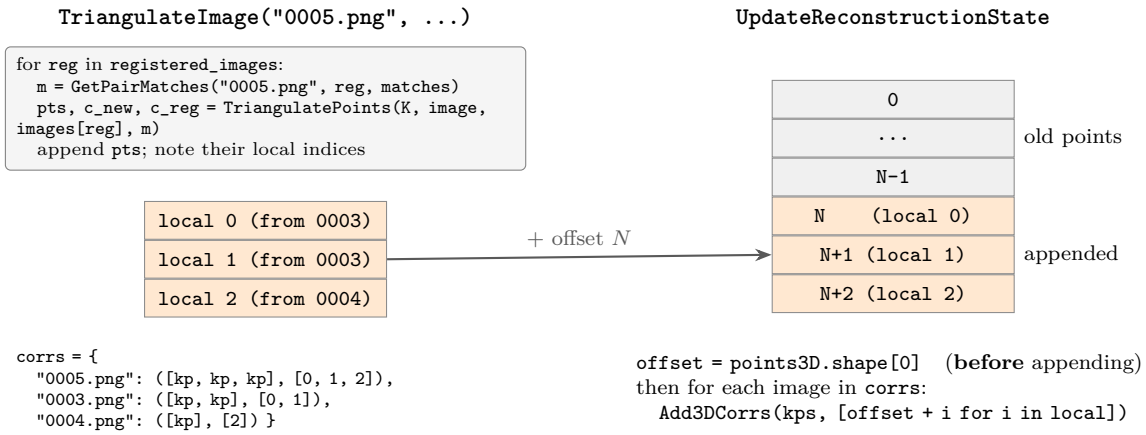


Figure 3: `TriangulateImage` numbers new points from 0. `UpdateReconstructionState` puts them at the end of the global array, so every local index is shifted by the old point count N.

The new image takes part in every pair, so its entry in `corrs` must *grow* across the loop rather than be overwritten.

6 Function reference

`impl/sfm/io.py` and `impl/util.py` – loading and coordinate helpers

Function	Input	Output	What it does
`ReadFeatureMatches`	`image_pair` (n1, n2) with n1 < n2; `data_folder`	$(M, 2)$ int array	reads `matches/n1-n2.txt`
`ReadKMatrix`	`data_folder`	`K` $(3, 3)$	reads `images/K.txt`
`MakeHomogeneous`	`pts`, `ax`	same array with a row/column of 1s appended	$[x, y] \to [x, y, 1]$. Points in rows $\Rightarrow$ `ax=1`
`HNormalize`	`pts`, `ax`	divides by last coordinate and drops it	$[x, y, w] \to [x/w, y/w]$

`impl/sfm/corrs.py` – bookkeeping between images and 3D points

Function	Input	Output	What it does
`GetPairMatches`	names `im1`, `im2`; `matches` dict	$(M, 2)$: col 0 = keypoint in `im1`, col 1 = in `im2`	looks up the pair in either order, flipping columns if needed
`Find2D3DCorrespondences`	new image name, `images`, `matches`, `registered_images`	`image_kp_idxs`, `p3D_idxs`: lists, length C	follows matches into registered images that already have 3D points; one entry per unique 3D point
`UpdateReconstruction State` Task 1.5	`new_points3D` $(L, 3)$, `corrs` dict, `points3D` $(N, 3)$, `images`	`points3D` $(N+L, 3)$, `images`	appends points, shifts local $\to$ global indices, calls `Add3DCorrs`

`impl/sfm/geometry.py` and `impl/dlt.py` – the geometry

Function	Input	Output	What it does
`EstimateEssentialMatrix` Task 1.1	`K`, `im1`, `im2` (`Image`), `matches` $(M, 2)$	`E` $(3, 3)$	8-point DLT on normalized keypoints, then enforce singular values $(1, 1, 0)$
`DecomposeEssentialMatrix`	`E`	list of 4 `(R, t)`	two rotations $\times$ two translation signs (cam 2 $\to$ cam 1)
`TriangulatePoints` Task 1.2	`K`, `im1`, `im2` *with poses set*, `matches`	`points3D` $(L, 3)$, `im1_corrs` $(L,)$, `im2_corrs` $(L,)$	DLT triangulation of not-yet-triangulated matches; removes points behind either camera
`EstimateImagePose` Task 1.4	`points2D` $(C, 2)$ px, `points3D` $(C, 3)$, `K`	`R` $(3, 3)$, `t` $(3,)$	absolute pose by DLT on normalized points
`TriangulateImage` Task 1.5	`K`, new `image_name`, `images`, `registered_images`, `matches`	new points $(L, 3)$ (local idx), `corrs` dict	triangulates new image against every registered image
`BuildProjection ConstraintMatrix`	`points2D` $(C, \geq 2)$, `points3D` $(C, 3)$	$(2C, 12)$	two DLT rows per 2D–3D pair (`dlt.py`)

`impl/sfm/vis.py` and `impl/vis.py` – plotting (all provided)

Function	Input	Output	What it does
`PlotImages`	`images` dict	figure	grid of all photos
`PlotWithKeypoints`	one `Image`	figure	photo with red keypoints
`PlotImagePairMatches`	`im1`, `im2`, `matches`	figure (**blocking**)	side-by-side photos with green match lines; 45 windows in `sfm.py`
`PlotCameras`	`images`, `registered_images`, `ax`	draws on `ax`	calls `PlotCamera` for each registered image
`Plot3DPoints`	`points` $(N, 3)$, `ax`	`ax`	black dots for the point cloud
`PlotCamera`	`R`, `t`, `ax`, `scale`, `color`	`ax`	camera frustum at centre $-R^\top t$

7 Conventions to keep straight

- **Pose** `(R, t)` in an `Image` maps *world* $\to$ *camera*: $X_{\text{cam}} = RX_{\text{world}} + t$. The camera centre is $-R^\top t$.

- **World frame** = camera of image 0003 (identity pose).
- **Essential matrix**: $\hat{x}_1^\top E\,\hat{x}_2 = 0$ (image 1 on the left), matching the `assert`.
- **Depth test**: a point is in front of a camera if $Z > 0$ after $RX + t$.
- **Normalized coordinates**: $\hat{x} = K^{-1}[u, v, 1]^\top$, the viewing-ray direction with the camera's internal parameters removed.
- **Array layout**: one point per *row* everywhere (`kps`, `points3D`, matches), so use `ax=1` with the `util.py` helpers and transpose around matrix products: `(A @ pts.T).T`.

8 Part 2: `line_fitting.py`

